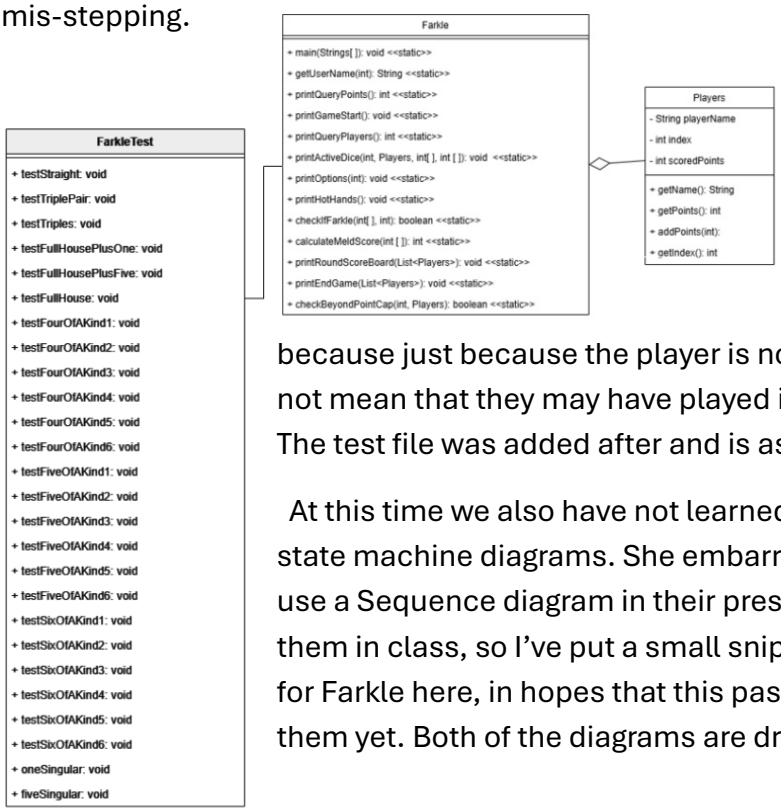
Ellen Hill – CPSC 224 – Homework 3: Zag Farkle Full Game & Multiplayer

The intent of this project was to build upon our work from the first homework and second homework, and implement hot seat multiplayer. We also added the Full house combo, a play to total points, and a round by round score board. This was my first time working with classes in Java, and I was very happy to find out that there is no importing- as long as the files are in the same directory, the classes are linked.

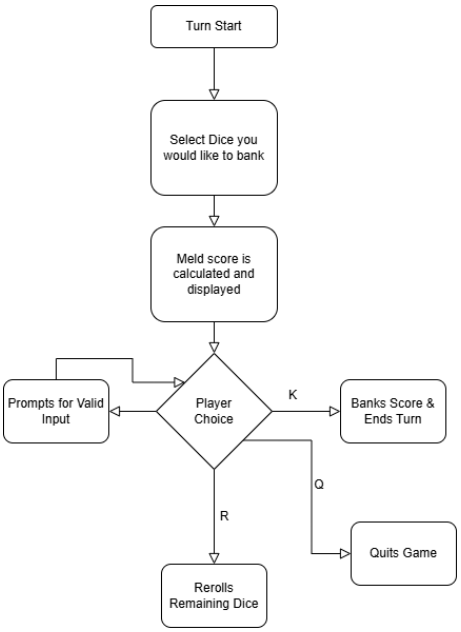
I am developing my coding style to start by breaking things up into methods and then implementing them as I go rather than writing them all in main. In a similar fashion, I think I will enjoy using classes much more now that I know they do not need to be packaged like in C++. I went about implementing full house first and it gave me quite a bit of trouble for the edge cases where the hands were primarily 5s and 1s, but I was able to get that figured out. I then went on to implementing the players classes and working on the multiplayer and point cap limitations.

This project was extended so that we could write our unit tests. While she did not cover the exact implementation of testing in our code, several of my classmates banded together to learn how to do them and implement them properly. I tested all variations of my scoring function and even found a bug that I didn't anticipate! We worked through the math calculation on paper, and then were able to find where in our codes the calculation was mis-stepping.



I only used two classes and compartmentalized the rest of my code into methods in Farkle.java, and kept the players in Players.java. I chose to represent them with aggregation because just because the player is not actively playing the game does not mean that they may have played in the past or are not still around. The test file was added after and is associated with the class.

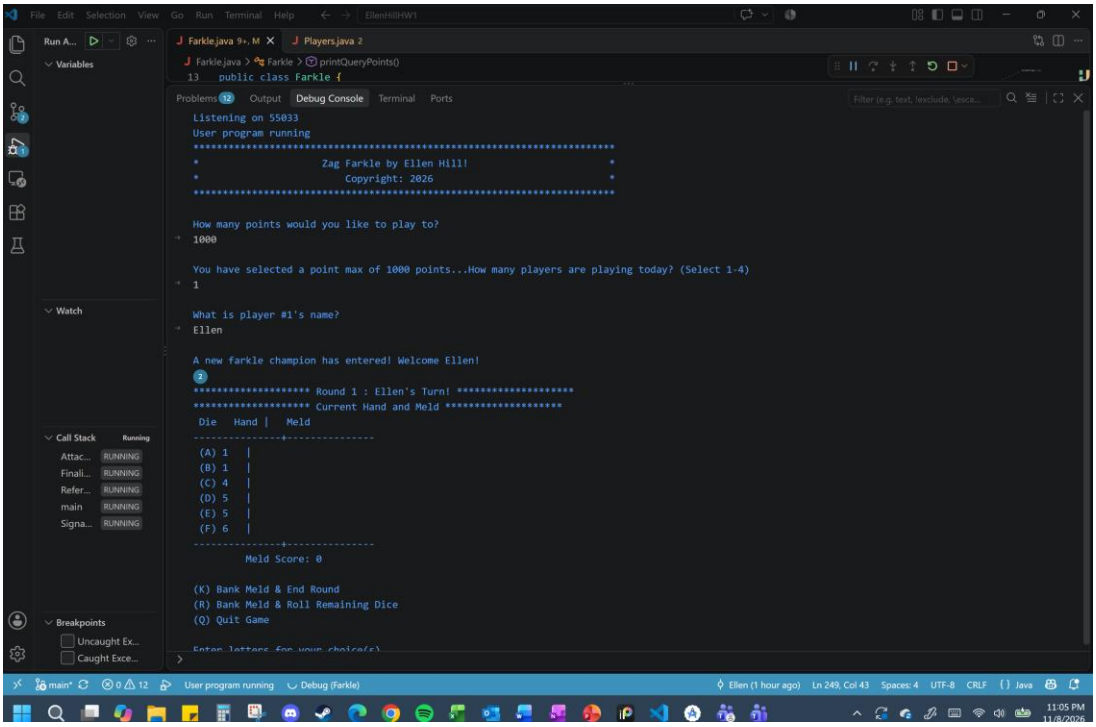
At this time we also have not learned UML Sequence diagrams, only state machine diagrams. She embarrassed a kid in class who tried to use a Sequence diagram in their presentation when we hadn't learned them in class, so I've put a small snippet of the State Machine Diagram for Farkle here, in hopes that this passes since we haven't learned them yet. Both of the diagrams are drawn with draw.io!



I really struggled with adding the fullhouse code just because of the edge cases, and wrapping up a nice clean farkle “end game” is much harder than it looks. I ended up writing this document before I took one more attempt at the “end game” so it is working in my submission, know I was successful. If I had more time, I probably would have played around with classes more since they were new to me and cleaned up main more.

Appendix:

Intro Screen:



End of Game:

```
****************** Round 4 : Ellen's Turn! ******************
****************** Current Hand and Meld ******************
 Die   Hand |   Meld
--------------+--------------
 (A)     |     1
 (B)     |     2
 (C)     |     2
 (D)     |     2
 (E)     |     5
 (F) 6   |
--------------+--------------
        Meld Score: 350

(K) Bank Meld & End Round
(R) Bank Meld & Roll Remaining Dice
(Q) Quit Game

Enter letters for your choice(s)
K

User program finished

Round over. Your score is currently: 1050

********************* SCORE BOARD **********************
     Ellen: 1050
********************************************************
You scored above the point cap! That's Game!

* Congratulations Ellen won with 1050! *
********************* SCORE BOARD **********************
     Ellen: 1050
********************************************************
```

Luckiest FullHouse that was also a hot hands:



Hitting Enter 3 Times:



Custom Naming:

